

Pràctica 3: Shallow Networks (1-D-1)

Laura Rodríguez Cima – Víctor Navas Portella

Grau d'Estadística Aplicada

Curs 2025/2026

Objectiu

En aquesta pràctica programareu (sense llibreries externes de DL) una **shallow network 1-D-1** amb una capa oculta d'amplada D .

Què farem avui?

- Implementar funció d'activació (ReLU).
- Inicialitzar pesos i biaixos amb `numpy`.
- Programar el **forward pass** (propagació cap endavant).
- Simular la sortida $\hat{y}(x)$ en un rang d'entrada i visualitzar-la.
- Calcular els **nodes** i el nombre de **regions lineals**.

Imports necessaris per a la pràctica

```
import numpy as np
import matplotlib.pyplot as plt
```

L'Arquitectura del nostre model

Treballarem amb una xarxa **1-D-1**:

- **Entrada (x):** Un únic valor escalar.
- **Capa Oculta:** D neurones amb activació no lineal.
- **Sortida ($\hat{y}$):** Una combinació lineal de les activacions.

Equacions de la xarxa

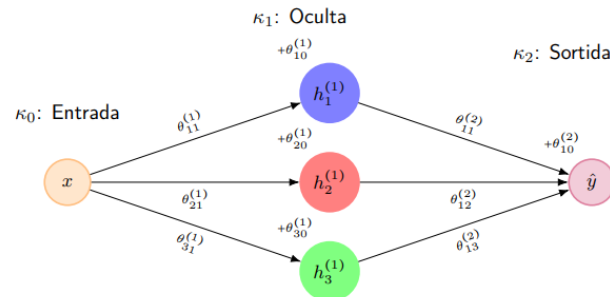
1. Capa oculta: $z^{(1)} = \Theta^{(1)}x + \theta_0^{(1)}$
2. Activació: $h^{(1)} = a^{(1)}(z^{(1)})$
3. Sortida: $\hat{y} = a^{(2)}\left(\Theta^{(2)}h^{(1)} + \theta_0^{(2)}\right)$

En aquesta pràctica fixarem $a^{(2)}$ com a **lineal** i treballarem amb $a^{(1)}$ ReLU.

Arquitectura 1-3-1 (referència visual)

A teoria, l'exemple 1-3-1 mostra clarament les dimensions:

- $\Theta^{(1)} \in \mathbb{R}^{3 \times 1}$, $\theta_0^{(1)} \in \mathbb{R}^{3 \times 1}$
- $\Theta^{(2)} \in \mathbb{R}^{1 \times 3}$, $\theta_{10}^{(2)} \in \mathbb{R}^{1 \times 1}$



Capa oculta (κ_1):

$$\mathbf{h}^{(1)} = a(\Theta^{(1)}x + \theta_0^{(1)})$$

Amb $\Theta^{(1)} \in \mathbb{R}^{3 \times 1}$

Capa sortida (κ_2):

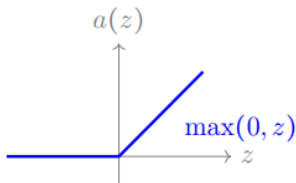
$$\hat{y} = a(\Theta^{(2)}\mathbf{h}^{(1)} + \theta_{10}^{(2)})$$

Amb $\Theta^{(2)} \in \mathbb{R}^{1 \times 3}$

Pas 1: Definició de l'activació

La xarxa només pot aprendre funcions complexes si introduïm una **no-linealitat**. Usarem ReLU (Rectified Linear Unit):

$$a(z) = \max(0, z)$$



Tasca: Implementa la funció següent usant numpy:

- `relu(z)`: Retorna 0 si $z < 0$, i z si $z \geq 0$. Útil: `np.maximum(0, z)`.

```
def relu(z):  
    return .....
```

Pas 2: Inicialització de Pesos i Biaixos

Per construir una xarxa amb D neurones ocultes, hem de definir 4 estructures de dades. Utilitzarem `np.random.uniform(min, max, shape)`.

1. Capa Oculta (Hidden Layer)

- **Pesos ($\Theta^{(1)}$):** Matriu de $(D \times 1)$. Cada fila és el pes d'una neurona per a l'única entrada x .
- **Biaixos ($\theta_0^{(1)}$):** Vector columna de $(D \times 1)$. Un biaix per a cada neurona.

2. Capa de Sortida (Output Layer)

- **Pesos ($\Theta^{(2)}$):** Matriu de $(1 \times D)$. Una fila que pondera les D sortides de la capa anterior.
- **Biaix ($\theta_0^{(2)}$):** Un escalar o matriu de (1×1) .

Tasca: Prova a generar aquestes matrius amb valors aleatoris. *Exemple per a $\Theta^{(1)}$:*

```
Theta1 = np.random.uniform(-5, 5, (D, 1))
```

Pas 3: El Forward Pass (La propagació)

Tasca: Programa la funció `forward_pass(x, Theta1, theta0_1, Theta2, theta0_2, activation_fn)` que calculi la sortida per a qualsevol entrada.

```
def forward_pass(x, Theta1, theta0_1, Theta2, theta0_2, activation_fn):  
  
    # 1) Pre-activació capa oculta:  $z1 = \text{Theta1} * x + \text{theta0}_1$   
    z1 = .....  
  
    # 2) Activació capa oculta amb activation_fn  
    h1 = .....  
  
    # 3) Sortida lineal:  $y\_hat = \text{Theta2} * h1 + \text{theta0}_2$   
    y_hat = .....  
  
    # 4) Retornem vector simple  
    return .....
```

Instruccions:

- 1 Utilitza `np.dot(Theta1, [x])` per calcular la pre-activació $z^{(1)}$ a la capa oculta. L'ús de `[x]` converteix la teva llista de dades en una matriu fila compatible amb els pesos.
- 2 Aplica la funció d'activació escollida sobre el resultat.
- 3 Realitza el segon producte de matrius amb `Theta2` i suma el biaix de sortida.
- 4 **Consell:** Finalitza amb `.flatten()` per retornar un vector simple de resultats.

Pas 4: Inicialització i Simulació

Per visualitzar la xarxa, necessitem valors concrets per als pesos i un vector concret per a les dades d'entrada.

Tasca: Crea una funció de simulació `simulate_shallow_network(D, activation_fn=relu)`.

Instruccions:

- 1 Defineix una llavor per a reproductibilitat dels resultats (41).
- 2 Inicialitza els pesos i els biaixos de la capa oculta i la capa de sortida com explicat en el Pas 2.
- 3 Crea un vector d'entrades `x_vals` amb `np.linspace(-5, 5, 1000)`.
- 4 Crida el teu `forward_pass` per obtenir la corba de la xarxa.

Pas 4: Inicialització i Simulació

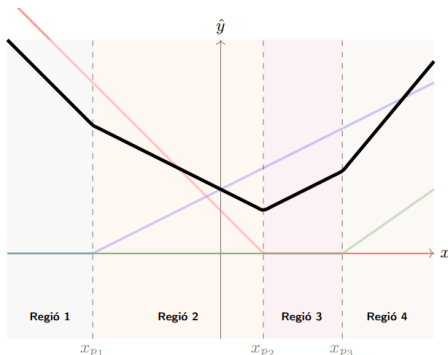
```
def simulate_shallow_network(D, activation_fn=relu):  
  
    # 1) Llavor per a reproductibilitat  
    np.random.seed(41)  
  
    # 2) Inicialització de pesos i biaixos com indicat al Pas 2  
  
    # Capa oculta (D neurones)  
    # Matriu de pesos de la capa oculta  
    Theta1 = .....  
    # Biaixos de la capa oculta  
    theta0_1 = .....  
  
    # Capa de sortida (1 neurona)  
    # Matriu de pesos de la capa de sortida  
    Theta2 = .....  
    # Biaix de la capa de sortida  
    theta0_2 = .....  
  
    # 3) Dades d'entrada  
    x_vals = .....  
  
    # 4) Forward pass  
    y_hat = .....  
  
    return x_vals, y_hat
```

Pas 5: Nodes i regions lineals (interpretació geomètrica)

Cada neurona ReLU de la capa oculta introduceix un **punt d'inflexió**:

$$\theta_{i1}^{(1)}x + \theta_{i0}^{(1)} = 0 \quad \Rightarrow \quad x_{p_i} = -\frac{\theta_{i0}^{(1)}}{\theta_{i1}^{(1)}}.$$

Aquests punts divideixen l'eix x en **regions lineals**. Amb D neurones, en 1D el màxim és $D + 1$ **regions**. Per exemple, la combinació de 3 neurones ReLU crea una funció amb 3 plecs (3 nodes) i 4 regions lineals. A mesura que afegim neurones, podem aproximar corbes cada cop més complexes.



Pas 5: Nodes i regionals lineals

Tasca: Crea una funció `get_theoretical_nodes(Theta1, theta0_1)` que implementi el càlcul vectoritzat dels nodes.

```
def get_theoretical_nodes(Theta1, theta0_1):  
    """  
    Retorna els x on cada neurona s'activa/apaga:  $z = w * x + b = 0$   
    Pesos: Theta1  
    Biaixos: theta0_1  
    """  
    # nodes_x = -b / w (vectoritzats amb .flatten())  
    nodes_x = .....  
  
    # Ordenem per pintar-los bé sobre l'eix x  
    return np.sort(nodes_x)
```

Pas 5: Nodes i regionals lineals

Tasca: Amplia la teva funció de simulació de la xarxa `simulate_shallow_network(D, activation_fn=relu)` per tal que retorni també els nodes.

```
def simulate_shallow_network(D, activation_fn=relu):  
  
    np.random.seed(41)  
  
    Theta1 = .....  
    theta0_1 = .....  
    Theta2 = .....  
    theta0_2 = .....  
  
    x_vals = .....  
  
    y_hat = .....  
  
    # Càlcul dels nodes (punts amb coordenades (x,y))  
    # Coordenada x (on es produeix el canvi de pendent) calculada amb la funció prè  
    #   via  
    nodes_x = .....  
    # Coordenada y (valor de la funció de la xarxa en els punts x)  
    nodes_y = forward_pass(nodes_x, Theta1, theta0_1, Theta2, theta0_2, activation_fn  
        )  
  
    return x_vals, y_hat, nodes_x, nodes_y
```

Pas 6: Visualització dels resultats per a diferents valors de D

```
# Visualització de l'evolució segons el nombre de neurones de la capa intermitja
# casos D = 2, 4, 8 i 50, proveu-ne més si teniu temps

n_list = [2, 4, 8, 50]

plt.figure(figsize=(12, 8))

results_n = []
results_params = []
results_regions = []

for i, D in enumerate(n_list):

    x, y, nx, ny = simulate_shallow_network(D, activation_fn=relu)

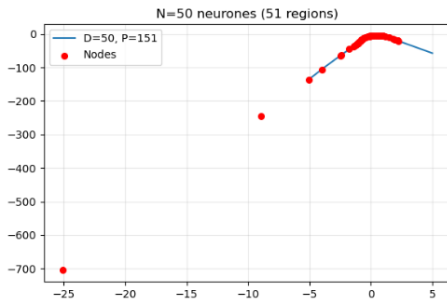
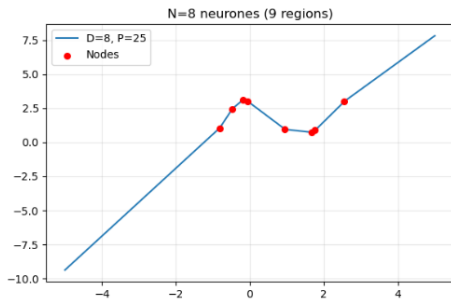
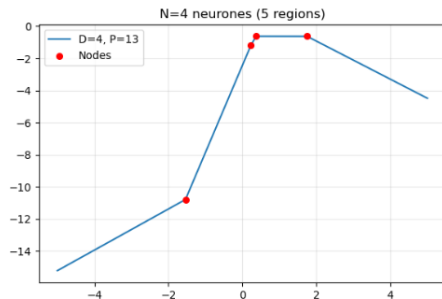
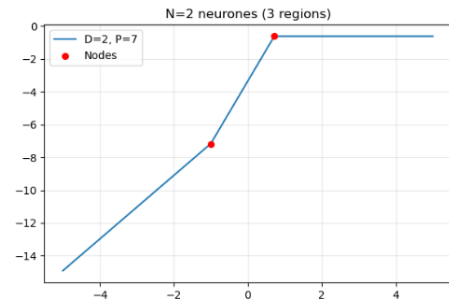
    params = (D * 1) + D + (1 * D) + 1
    regions = len(nx) + 1

    results_n.append(D)
    results_params.append(params)
    results_regions.append(regions)

    plt.subplot(2, 2, i + 1)
    plt.plot(x, y, label=f'D={D}, P={params}')
    plt.scatter(nx, ny, color='red', s=30, zorder=5, label='Nodes')
    plt.title(f'N={D} neurones ({regions} regions)')
    plt.grid(True, alpha=0.3)
    plt.legend()

plt.tight_layout()
plt.show()
```

Pas 6: Visualització dels resultats per a diferents valors de D



Després d'executar el vostre codi per a $D = [2, 4, 8, 50]$:

- Per què algunes neurones no generen una regió visible?
- Què passa amb la forma de la funció quan els pesos Θ_2 són molt grans? I si són propers a zero?
- És possible que dues neurones diferents generin el mateix node? Què passaria amb el recompte de regions?

BONUS: altres funcions d'activació

Tasca: utilitzeu la funció perceptró mostrada a continuació com a funció d'activació enlloc de ReLU, i compareu els resultats amb els obtinguts abans.

```
def perceptron(z):  
    return np.where(z >= 0, 1, 0)
```

BONUS: visualització dels resultats amb perceptró

